

DL_01_fundamentos

March 13, 2026

1 Deep Learning: Fundamentos Teóricos

1.1 Introducción

Este notebook constituye una introducción completa a los fundamentos del Deep Learning. Está diseñado para ser seguido en clase, combinando explicaciones teóricas detalladas con ejemplos prácticos en código.

1.1.1 Objetivos de aprendizaje

Al finalizar este notebook, serás capaz de:

1. Explicar qué es el Deep Learning y cómo se diferencia del Machine Learning tradicional
2. Describir la estructura y funcionamiento de una neurona artificial (perceptrón)
3. Comprender el papel de las funciones de activación y por qué son necesarias
4. Entender el proceso de forward propagation y backpropagation
5. Seleccionar la función de pérdida adecuada según el tipo de problema

1.1.2 Contenido

1. ¿Qué es Deep Learning?
 2. El Perceptrón: La Neurona Artificial
 3. Funciones de Activación
 4. Redes Neuronales Multicapa
 5. Forward Propagation
 6. Backpropagation
 7. Funciones de Pérdida
-

1.2 1. ¿Qué es Deep Learning?

1.2.1 Definición

Deep Learning (Aprendizaje Profundo) es una rama del Machine Learning que utiliza **redes neuronales artificiales** con múltiples capas para aprender representaciones jerárquicas de los datos. El término “profundo” hace referencia a la cantidad de capas que componen estas redes.

1.2.2 Contexto histórico

Aunque las bases teóricas de las redes neuronales se establecieron en los años 1940-1960, el Deep Learning moderno despegó alrededor de 2012 gracias a tres factores convergentes:

1. **Disponibilidad de grandes datasets:** Internet permitió recopilar millones de imágenes, textos y otros datos etiquetados.
2. **Poder computacional (GPUs):** Las tarjetas gráficas, diseñadas originalmente para videojuegos, resultaron ideales para el cálculo matricial masivo que requieren las redes neuronales.
3. **Avances algorítmicos:** Nuevas técnicas como ReLU, Dropout, Batch Normalization y mejores inicializaciones de pesos resolvieron problemas que habían estancado el campo durante décadas.

1.2.3 ¿En qué se diferencia del Machine Learning tradicional?

La diferencia fundamental radica en cómo se procesan los datos de entrada:

Machine Learning Tradicional

Datos brutos → [INGENIERÍA DE FEATURES manual] → Features → [Modelo simple] → Predicción

En el ML tradicional, un experto humano debe diseñar manualmente las características (features) relevantes. Por ejemplo, para reconocer dígitos escritos a mano:

- Contar píxeles negros en cada cuadrante
- Detectar la presencia de líneas horizontales/verticales
- Medir la proporción alto/ancho
- Calcular el centro de masa de los trazos

Este proceso requiere conocimiento del dominio y es laborioso.

Deep Learning

Datos brutos → [RED NEURONAL aprende features automáticamente] → Predicción

En Deep Learning, la red aprende automáticamente qué características son relevantes. Las primeras capas detectan patrones simples (bordes, colores), las capas intermedias combinan estos patrones en conceptos más complejos (texturas, formas), y las capas finales reconocen objetos completos.

1.2.4 El concepto de “profundidad”

¿Por qué usar múltiples capas? La respuesta está en la **composicionalidad**: conceptos complejos se construyen a partir de conceptos más simples.

Imaginemos una red para reconocer rostros:

Capa 1: Detecta bordes y gradientes de intensidad
→ líneas horizontales, verticales, diagonales

Capa 2: Combina bordes en texturas y patrones simples
→ texturas de piel, patrones de cabello

Capa 3: Detecta partes de objetos
→ ojos, nariz, boca, orejas

Capa 4: Combina partes en objetos completos
→ rostros completos, expresiones

Capa 5: Reconoce identidades o atributos
→ "Esta es la persona X", "Expresión: sonriente"

Esta jerarquía de representaciones es lo que hace al Deep Learning tan poderoso: no necesitamos decirle a la red qué es un “ojo” o una “nariz”, ella descubre estos conceptos por sí misma durante el entrenamiento.

1.3 2. El Perceptrón: La Neurona Artificial

1.3.1 Inspiración biológica

El perceptrón está inspirado en las neuronas biológicas del cerebro humano. Una neurona biológica:

1. **Recibe señales** de otras neuronas a través de las dendritas
2. **Procesa** estas señales en el soma (cuerpo celular)
3. Si la señal acumulada supera un umbral, **dispara** una señal eléctrica
4. Esta señal viaja por el axón hacia otras neuronas

El perceptrón artificial replica este comportamiento de forma simplificada.

1.3.2 Estructura matemática del perceptrón

Un perceptrón realiza dos operaciones fundamentales:

Paso 1: Combinación lineal (suma ponderada)

$$z = \sum_{i=1}^n w_i \cdot x_i + b = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

O en notación vectorial:

$$z = \mathbf{w}^T \mathbf{x} + b$$

Paso 2: Función de activación

$$y = f(z)$$

Donde: - $\mathbf{x} = [x_1, x_2, \dots, x_n]$: **Vector de entrada** (los datos) - $\mathbf{w} = [w_1, w_2, \dots, w_n]$: **Pesos** (parámetros aprendibles que determinan la importancia de cada entrada) - b : **Sesgo o bias** (permite desplazar la función de activación) - f : **Función de activación** (introduce no-linealidad) - y : **Salida** del perceptrón

1.3.3 Interpretación geométrica

Los pesos y el bias definen un **hiperplano** en el espacio de entrada. Este hiperplano divide el espacio en dos regiones, permitiendo clasificar puntos según de qué lado del hiperplano caen.

Para dos dimensiones (dos entradas x_1, x_2):

$$w_1x_1 + w_2x_2 + b = 0$$

Esta es la ecuación de una recta. Los puntos donde $w_1x_1 + w_2x_2 + b > 0$ están en un lado (clase 1), y los puntos donde $w_1x_1 + w_2x_2 + b < 0$ están en el otro (clase 0).

1.3.4 El papel del bias

El bias b es crucial porque permite que el hiperplano de decisión no pase obligatoriamente por el origen. Sin bias:

- La recta $w_1x_1 + w_2x_2 = 0$ siempre pasa por $(0, 0)$
- Esto limita severamente qué problemas podemos resolver

Con bias:

- Podemos desplazar la recta a cualquier posición
- Ganamos un grado de libertad adicional

```
[1]: import numpy as np
import matplotlib.pyplot as plt

class Perceptron:
    """
    Implementación del Perceptrón clásico (Rosenblatt, 1958).

    El perceptrón es el bloque de construcción básico de las redes neuronales.
    Aunque simple, puede resolver problemas linealmente separables.
    """

    def __init__(self, n_inputs):
        """
        Inicializa el perceptrón con pesos aleatorios pequeños.

        ¿Por qué pesos pequeños? Si los pesos son muy grandes, la salida
        de la función de activación estará saturada desde el inicio,
        dificultando el aprendizaje.

        Parámetros:
            n_inputs: Número de entradas que recibirá el perceptrón
        """
        # Inicialización con valores aleatorios pequeños (distribución normal)
        self.weights = np.random.randn(n_inputs) * 0.01
        self.bias = 0.0
```

```

def forward(self, x):
    """
    Calcula la salida del perceptrón para una entrada dada.

    Proceso:
    1. Calcular suma ponderada:  $z = w \cdot x + b$ 
    2. Aplicar función de activación escalón:  $y = 1$  si  $z > 0$ , sino 0

    Parámetros:
        x: Vector de entrada (numpy array)

    Retorna:
        Salida binaria (0 o 1)
    """
    # Paso 1: Combinación lineal
    z = np.dot(self.weights, x) + self.bias

    # Paso 2: Función de activación escalón (step function)
    # Esta es la función de activación original del perceptrón
    return 1 if z > 0 else 0

def train(self, X, y, epochs=100, learning_rate=0.1):
    """
    Entrena el perceptrón usando la regla de aprendizaje del perceptrón.

    La regla es simple:
    - Si la predicción es correcta: no hacer nada
    - Si la predicción es incorrecta: ajustar pesos en la dirección del
    ↪ error

    Matemáticamente:
         $w_{new} = w_{old} + learning\_rate * error * x$ 
         $b_{new} = b_{old} + learning\_rate * error$ 

    Donde  $error = y_{real} - y_{predicho}$ 

    Parámetros:
        X: Matriz de datos de entrada (n_samples, n_features)
        y: Vector de etiquetas (n_samples,)
        epochs: Número máximo de pasadas por los datos
        learning_rate: Tasa de aprendizaje (cuánto ajustar los pesos)
    """
    for epoch in range(epochs):
        total_errors = 0

        for xi, yi in zip(X, y):

```

```

        # Hacer predicción
        prediction = self.forward(xi)

        # Calcular error
        error = yi - prediction

        # Actualizar pesos solo si hay error
        if error != 0:
            self.weights += learning_rate * error * xi
            self.bias += learning_rate * error
            total_errors += 1

        # Si no hay errores, el perceptrón ha convergido
        if total_errors == 0:
            print(f"Convergencia alcanzada en época {epoch + 1}")
            return

    print(f"Entrenamiento completado. Errores finales: {total_errors}")

```

1.3.5 Ejemplo práctico: Aprendiendo la compuerta AND

Vamos a entrenar un perceptrón para aprender la función lógica AND. Esta es la tabla de verdad:

x_1	x_2	AND
0	0	0
0	1	0
1	0	0
1	1	1

La compuerta AND es **linealmente separable**: podemos dibujar una línea que separe los casos donde la salida es 0 de los casos donde es 1.

```

[2]: # Datos de la compuerta AND
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

y = np.array([0, 0, 0, 1]) # Salida de AND

# Crear y entrenar el perceptrón
perceptron = Perceptron(n_inputs=2)
print("Pesos iniciales:", perceptron.weights)
print("Bias inicial:", perceptron.bias)
print()

```

```

perceptron.train(X, y, epochs=100, learning_rate=0.1)

print()
print("Pesos finales:", perceptron.weights)
print("Bias final:", perceptron.bias)

# Verificar que funciona
print("\n=== Verificación ===")
for xi, yi in zip(X, y):
    pred = perceptron.forward(xi)
    estado = " " if pred == yi else " "
    print(f"{xi} → Predicción: {pred}, Real: {yi} {estado}")

```

Pesos iniciales: [0.00365915 0.00873498]

Bias inicial: 0.0

Convergencia alcanzada en época 8

Pesos finales: [0.20365915 0.10873498]

Bias final: -0.30000000000000004

=== Verificación ===

```

[0 0] → Predicción: 0, Real: 0
[0 1] → Predicción: 0, Real: 0
[1 0] → Predicción: 0, Real: 0
[1 1] → Predicción: 1, Real: 1

```

```

[3]: # Visualizar la frontera de decisión aprendida
plt.figure(figsize=(8, 6))

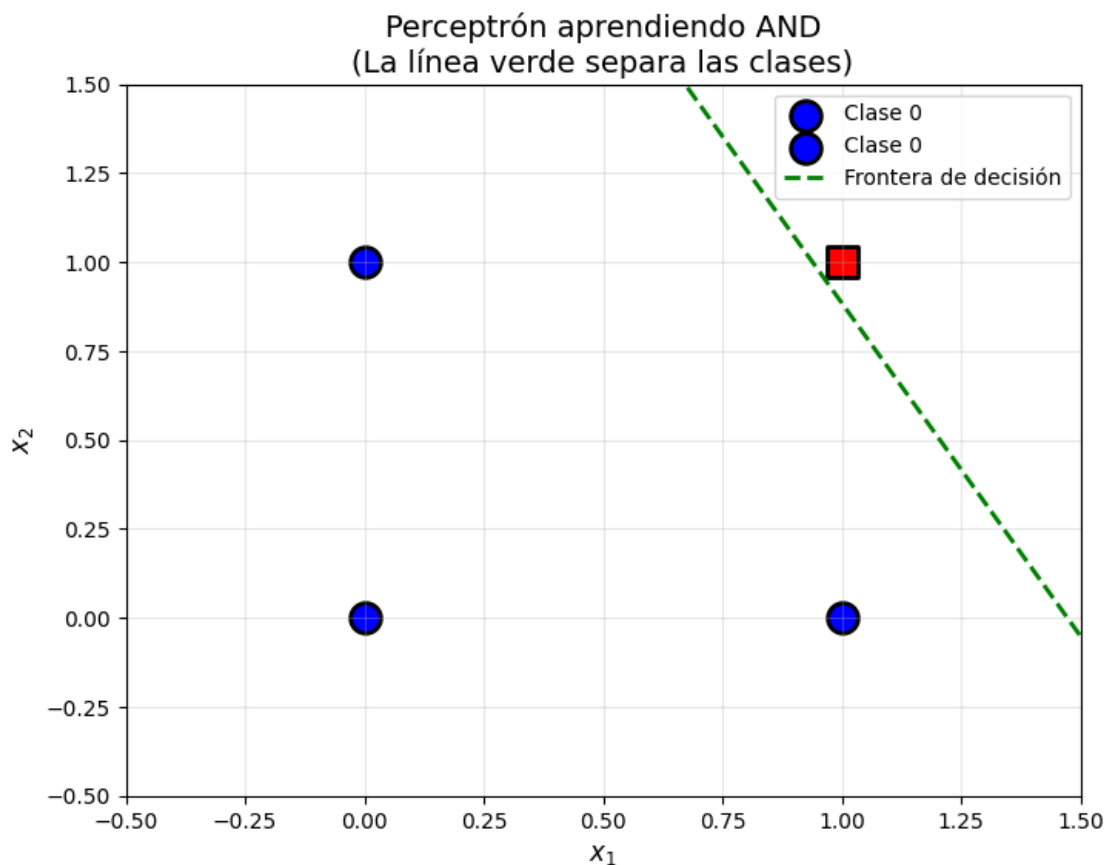
# Graficar los puntos de datos
for i, (xi, yi) in enumerate(zip(X, y)):
    color = 'blue' if yi == 0 else 'red'
    marker = 'o' if yi == 0 else 's'
    plt.scatter(xi[0], xi[1], c=color, marker=marker, s=200,
                label=f'Clase {yi}' if i < 2 else None, edgecolors='black',
                linewidth=2)

# Dibujar la frontera de decisión:  $w_1x_1 + w_2x_2 + b = 0$ 
# Despejando  $x_2$ :  $x_2 = -(w_1x_1 + b) / w_2$ 
if perceptron.weights[1] != 0:
    x1_line = np.linspace(-0.5, 1.5, 100)
    x2_line = -(perceptron.weights[0] * x1_line + perceptron.bias) / perceptron.
    weights[1]
    plt.plot(x1_line, x2_line, 'g--', linewidth=2, label='Frontera de decisión')

```

```
plt.xlim(-0.5, 1.5)
plt.ylim(-0.5, 1.5)
plt.xlabel('$x_1$', fontsize=12)
plt.ylabel('$x_2$', fontsize=12)
plt.title('Perceptr3n aprendiendo AND\n(La l3nea verde separa las clases)',
         ↪ fontsize=14)
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

print(f"Ecuaci3n de la frontera: {perceptron.weights[0]:.2f}·x + {perceptron.
     ↪ weights[1]:.2f}·x + {perceptron.bias:.2f} = 0")
```



Ecuaci3n de la frontera: $0.20 \cdot x + 0.11 \cdot x + -0.30 = 0$

1.3.6 Limitaci3n fundamental: El problema XOR

Un solo perceptr3n solo puede resolver problemas **linealmente separables**. Esto significa que debe existir una l3nea (o hiperplano en dimensiones superiores) que separe perfectamente las clases.

La funci3n XOR (OR exclusivo) **no es linealmente separable**:

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

No existe ninguna línea recta que pueda separar los puntos (0,0) y (1,1) de los puntos (0,1) y (1,0).

Esta limitación, demostrada por Minsky y Papert en 1969, causó el primer “invierno de la IA”, un período de reducción drástica en la financiación de investigación en redes neuronales.

La solución vino con las **redes multicapa**: combinando múltiples perceptrones, podemos resolver problemas no linealmente separables. Pero primero, necesitamos entender las funciones de activación.

1.4 3. Funciones de Activación

1.4.1 ¿Por qué necesitamos funciones de activación?

Las funciones de activación introducen **no-linealidad** en la red. Esta es una necesidad matemática fundamental, no una elección arbitraria.

Demostración: Sin activación, las capas son inútiles Consideremos una red de dos capas sin funciones de activación:

- Capa 1: $\mathbf{h} = W_1\mathbf{x} + \mathbf{b}_1$
- Capa 2: $\mathbf{y} = W_2\mathbf{h} + \mathbf{b}_2$

Sustituyendo:

$$\mathbf{y} = W_2(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$$

$$\mathbf{y} = W_2W_1\mathbf{x} + W_2\mathbf{b}_1 + \mathbf{b}_2$$

$$\mathbf{y} = W'\mathbf{x} + \mathbf{b}'$$

Donde $W' = W_2W_1$ y $\mathbf{b}' = W_2\mathbf{b}_1 + \mathbf{b}_2$.

¡La red de dos capas es equivalente a una sola capa lineal! No importa cuántas capas apilemos: sin no-linealidad, la red solo puede aprender funciones lineales.

1.4.2 Propiedades deseables de una función de activación

1. **No-linealidad:** Permite aprender funciones complejas
2. **Diferenciabilidad:** Necesaria para calcular gradientes (backpropagation)
3. **Rango de salida apropiado:** Según la capa y el problema
4. **Eficiencia computacional:** Se evalúa millones de veces
5. **Gradientes bien comportados:** Evitar que desaparezcan o exploten

1.4.3 Principales funciones de activación

```
[4]: import torch
import torch.nn.functional as F

# Rango de valores para visualización
x = torch.linspace(-5, 5, 200)

# Calcular cada función de activación
activaciones = {
    'Sigmoid': torch.sigmoid(x),
    'Tanh': torch.tanh(x),
    'ReLU': F.relu(x),
    'LeakyReLU': F.leaky_relu(x, negative_slope=0.1),
    'ELU': F.elu(x),
    'GELU': F.gelu(x)
}

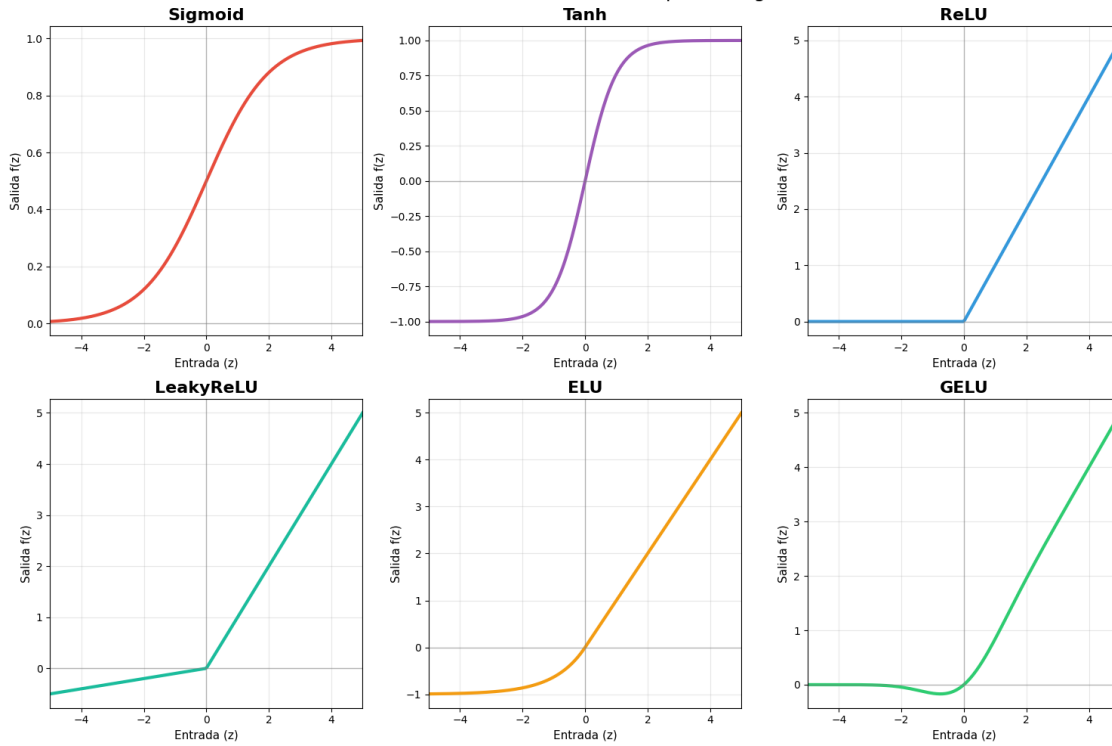
# Visualizar
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
axes = axes.flatten()

colores = ['#e74c3c', '#9b59b6', '#3498db', '#1abc9c', '#f39c12', '#2ecc71']

for idx, (nombre, y) in enumerate(activaciones.items()):
    ax = axes[idx]
    ax.plot(x.numpy(), y.numpy(), color=colores[idx], linewidth=3)
    ax.axhline(y=0, color='gray', linestyle='--', alpha=0.5, linewidth=1)
    ax.axvline(x=0, color='gray', linestyle='--', alpha=0.5, linewidth=1)
    ax.set_title(nombre, fontsize=16, fontweight='bold')
    ax.set_xlabel('Entrada (z)', fontsize=11)
    ax.set_ylabel('Salida f(z)', fontsize=11)
    ax.grid(True, alpha=0.3)
    ax.set_xlim(-5, 5)

plt.tight_layout()
plt.suptitle('Funciones de Activación en Deep Learning', fontsize=18, y=1.02)
plt.show()
```

Funciones de Activación en Deep Learning



1.4.4 Análisis detallado de cada función

1. Sigmoid (Logística)

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Características: - **Rango de salida:** $(0, 1) \rightarrow$ Interpretable como probabilidad - **Derivada:** $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, máximo de 0.25

Ventajas: - Salida interpretable como probabilidad - Suave y diferenciable en todo el dominio

Desventajas: - **Vanishing gradient:** Para $|z| > 4$, la derivada es prácticamente 0. Esto impide que los gradientes fluyan hacia atrás, haciendo que las capas iniciales aprendan muy lentamente. - **No centrada en cero:** La salida siempre es positiva, lo que puede causar problemas de optimización. - **Computacionalmente costosa:** Requiere calcular exponencial.

Uso actual: Principalmente en la capa de salida para **clasificación binaria**.

2. Tanh (Tangente Hiperbólica)

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = 2\sigma(2z) - 1$$

Características: - **Rango de salida:** $(-1, 1) \rightarrow$ Centrada en cero - **Derivada:** $\tanh'(z) = 1 - \tanh^2(z)$, máximo de 1

Ventajas: - Centrada en cero (mejor que sigmoid para capas ocultas) - Gradientes más fuertes que sigmoid

Desventajas: - Sigue sufriendo de vanishing gradient en los extremos - Computacionalmente costosa

Uso actual: Ocasionalmente en RNNs y LSTMs.

3. ReLU (Rectified Linear Unit)

$$\text{ReLU}(z) = \max(0, z)$$

Características: - **Rango de salida:** $[0, \infty)$ - **Derivada:** 1 si $z > 0$, 0 si $z < 0$ (indefinida en $z = 0$, pero se suele tomar 0)

Ventajas: - **Extremadamente simple y rápida:** Solo una comparación - **No hay vanishing gradient** para valores positivos - **Sparsity:** Muchas neuronas tienen salida 0, lo que actúa como regularización

Desventajas: - **“Dying ReLU”:** Si una neurona siempre recibe valores negativos, su gradiente siempre es 0 y nunca aprende. La neurona “muere”. - No centrada en cero

Uso actual: La más usada en capas ocultas de redes profundas.

4. Leaky ReLU

$$\text{LeakyReLU}(z) = \begin{cases} z & \text{si } z > 0 \\ \alpha z & \text{si } z \leq 0 \end{cases}$$

Donde α es un valor pequeño, típicamente 0.01 o 0.1.

Ventajas: - Resuelve el problema de “dying ReLU” - Mantiene gradiente pequeño pero no nulo para valores negativos

Variantes: - **PReLU (Parametric ReLU):** α es un parámetro aprendible - **ELU:** $f(z) = z$ si $z > 0$, $\alpha(e^z - 1)$ si $z \leq 0$

5. GELU (Gaussian Error Linear Unit)

$$\text{GELU}(z) = z \cdot \Phi(z)$$

Donde $\Phi(z)$ es la función de distribución acumulativa de la distribución normal estándar.

Características: - Suave, no tiene esquinas como ReLU - Permite valores ligeramente negativos

Uso actual: Muy popular en **Transformers** (BERT, GPT, etc.).

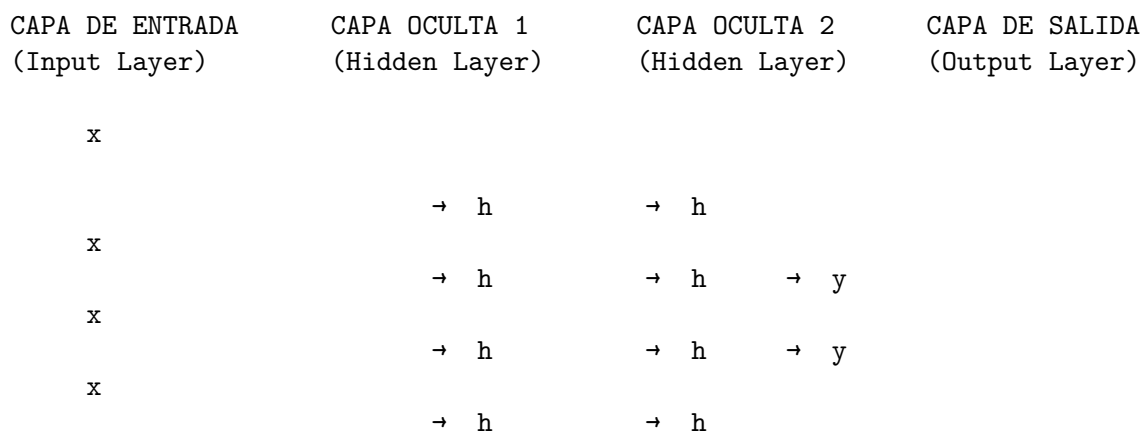
1.4.5 Guía de selección de funciones de activación

Ubicación	Problema	Recomendación
Capas ocultas	General	ReLU (primera opción)
Capas ocultas	Si hay dying ReLU	LeakyReLU o ELU
Capas ocultas	Transformers/NLP	GELU
Capa salida	Clasificación binaria	Sigmoid
Capa salida	Clasificación multiclase	Softmax
Capa salida	Regresión	Ninguna (lineal)
Capa salida	Regresión acotada $[0,1]$	Sigmoid
Capa salida	Regresión acotada $[-1,1]$	Tanh

1.5 4. Redes Neuronales Multicapa

1.5.1 Arquitectura del Perceptrón Multicapa (MLP)

Un **Multi-Layer Perceptron** (MLP) es una red neuronal con una o más capas ocultas entre la entrada y la salida.



1.5.2 Terminología

- **Capa de entrada:** No es realmente una “capa” de procesamiento; simplemente recibe los datos.
- **Capas ocultas:** Donde ocurre el procesamiento. Se llaman “ocultas” porque no vemos directamente sus salidas.
- **Capa de salida:** Produce la predicción final.
- **Red “fully connected” o “densa”:** Cada neurona está conectada a todas las neuronas de la capa anterior.
- **Profundidad:** Número de capas con parámetros aprendibles.
- **Anchura:** Número de neuronas en una capa.

1.5.3 Capacidad expresiva

El **Teorema de Aproximación Universal** (Cybenko, 1989; Hornik, 1991) establece que:

Una red neuronal con una sola capa oculta con suficientes neuronas puede aproximar cualquier función continua con precisión arbitraria.

Sin embargo, esto no significa que debamos usar redes poco profundas:

1. **Eficiencia:** Las redes profundas pueden representar las mismas funciones con exponencialmente menos neuronas.
2. **Generalización:** Las representaciones jerárquicas tienden a generalizar mejor.
3. **Aprendizaje:** En la práctica, las redes profundas son más fáciles de entrenar para problemas complejos.

```
[5]: import torch
import torch.nn as nn

class MLP(nn.Module):
    """
    Perceptrón Multicapa (Multi-Layer Perceptron).

    Esta clase implementa una red neuronal fully connected con:
    - Número configurable de capas ocultas
    - Activación ReLU entre capas
    - Sin activación en la salida (para usar con CrossEntropyLoss)
    """

    def __init__(self, input_size, hidden_sizes, output_size):
        """
        Construye la arquitectura de la red.

        Parámetros:
            input_size: Número de features de entrada
            hidden_sizes: Lista con el número de neuronas en cada capa oculta
                        Ejemplo: [256, 128, 64] crea 3 capas ocultas
            output_size: Número de neuronas de salida (clases en clasificación)
        """
        super().__init__() # Llamar al constructor de nn.Module

        # Construir las capas dinámicamente
        layers = []
        prev_size = input_size

        # Añadir cada capa oculta
        for hidden_size in hidden_sizes:
            # Capa lineal (fully connected)
            layers.append(nn.Linear(prev_size, hidden_size))
            # Función de activación
            layers.append(nn.ReLU())
            # Actualizar tamaño para la siguiente capa
            prev_size = hidden_size
```

```

# Capa de salida (sin activación, se aplica en la loss)
layers.append(nn.Linear(prev_size, output_size))

# nn.Sequential encadena las capas
self.network = nn.Sequential(*layers)

def forward(self, x):
    """
    Define el flujo de datos a través de la red.

    Este método es llamado automáticamente cuando hacemos:
    output = modelo(input)

    Parámetros:
        x: Tensor de entrada (batch_size, input_size)

    Retorna:
        Tensor de salida (batch_size, output_size)
    """
    return self.network(x)

# Ejemplo: Red para MNIST (imágenes 28x28 = 784 píxeles, 10 dígitos)
modelo = MLP(
    input_size=784,
    hidden_sizes=[256, 128, 64],
    output_size=10
)

print("Arquitectura de la red:")
print(modelo)

# Contar parámetros
total_params = sum(p.numel() for p in modelo.parameters())
print(f"\nTotal de parámetros: {total_params:,}")

# Desglose por capa
print("\nDesglose de parámetros:")
for name, param in modelo.named_parameters():
    print(f"    {name}: {param.numel():,} parámetros, shape {list(param.shape)}")

```

Arquitectura de la red:

```

MLP(
  (network): Sequential(
    (0): Linear(in_features=784, out_features=256, bias=True)
    (1): ReLU()
    (2): Linear(in_features=256, out_features=128, bias=True)

```

```

(3): ReLU()
(4): Linear(in_features=128, out_features=64, bias=True)
(5): ReLU()
(6): Linear(in_features=64, out_features=10, bias=True)
)
)

```

Total de parámetros: 242,762

Desglose de parámetros:

```

network.0.weight: 200,704 parámetros, shape [256, 784]
network.0.bias: 256 parámetros, shape [256]
network.2.weight: 32,768 parámetros, shape [128, 256]
network.2.bias: 128 parámetros, shape [128]
network.4.weight: 8,192 parámetros, shape [64, 128]
network.4.bias: 64 parámetros, shape [64]
network.6.weight: 640 parámetros, shape [10, 64]
network.6.bias: 10 parámetros, shape [10]

```

1.6 5. Forward Propagation

1.6.1 Definición

Forward propagation (propagación hacia adelante) es el proceso de pasar los datos de entrada a través de la red, capa por capa, hasta obtener una predicción en la salida.

1.6.2 Proceso matemático

Para una red de L capas, el forward pass consiste en aplicar secuencialmente:

Para cada capa $l = 1, 2, \dots, L$:

1. **Combinación lineal:**

$$\mathbf{z}^{[l]} = \mathbf{W}^{[l]} \mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}$$

2. **Activación:**

$$\mathbf{a}^{[l]} = f^{[l]}(\mathbf{z}^{[l]})$$

Donde: - $\mathbf{a}^{[0]} = \mathbf{x}$ (la entrada) - $\mathbf{W}^{[l]}$ es la matriz de pesos de la capa l - $\mathbf{b}^{[l]}$ es el vector de bias de la capa l - $f^{[l]}$ es la función de activación de la capa l - $\mathbf{a}^{[L]} = \hat{\mathbf{y}}$ es la predicción final

```

[6]: # Demostración visual del forward pass
import torch
import torch.nn.functional as F

print("=== FORWARD PASS PASO A PASO ===")
print()

# Entrada: batch de 2 muestras, 3 features cada una

```



```

x = torch.tensor([[1.0, 2.0, 3.0],
                  [4.0, 5.0, 6.0]])

print("ENTRADA (a = x):")
print(f"  Shape: {x.shape}")
print(f"  Datos:\n{x}")
print()

# ===== CAPA 1: 3 entradas + 4 neuronas =====
print(" " * 50)
print("CAPA 1: Lineal (3 → 4) + ReLU")
print(" " * 50)

# Pesos y bias (normalmente se inicializan aleatoriamente)
W1 = torch.tensor([[0.1, 0.2, 0.3],
                   [0.4, 0.5, 0.6],
                   [0.7, 0.8, 0.9],
                   [0.2, 0.1, 0.3]])
b1 = torch.tensor([0.1, -0.1, 0.2, 0.0])

print(f"W1 shape: {W1.shape} (out_features × in_features)")
print(f"b1 shape: {b1.shape}")

# Paso 1: Combinación lineal
z1 = torch.matmul(x, W1.T) + b1
print(f"\nz1 = x · W1 + b1:")
print(f"  Shape: {z1.shape}")
print(f"  Valores:\n{z1}")

# Paso 2: Activación ReLU
a1 = F.relu(z1)
print(f"\na1 = ReLU(z1):")
print(f"  Shape: {a1.shape}")
print(f"  Valores:\n{a1}")
print()

# ===== CAPA 2: 4 entradas + 2 neuronas (salida) =====
print(" " * 50)
print("CAPA 2: Lineal (4 → 2) - Capa de salida")
print(" " * 50)

W2 = torch.tensor([[0.2, 0.3, 0.1, 0.4],
                   [0.5, 0.2, 0.3, 0.1]])
b2 = torch.tensor([0.1, -0.1])

# Combinación lineal (sin activación en la salida para clasificación)
z2 = torch.matmul(a1, W2.T) + b2

```

```

print(f"\n $z^2 = a^1 \cdot W^2 + b^2$  (LOGITS):")
print(f"  Shape: {z2.shape}")
print(f"  Valores:\n{z2}")

# Para obtener probabilidades, aplicamos softmax
probs = F.softmax(z2, dim=1)
print(f"\nProbabilidades (softmax):")
print(f"  {probs}")
print(f"  Suma por fila: {probs.sum(dim=1)} (siempre suma 1)")

# Predicción final
predicciones = probs.argmax(dim=1)
print(f"\nPredicciones (clase con mayor probabilidad): {predicciones}")

```

=== FORWARD PASS PASO A PASO ===

ENTRADA ($a = x$):

Shape: torch.Size([2, 3])

Datos:

```

tensor([[1., 2., 3.],
        [4., 5., 6.]])

```

CAPA 1: Lineal (3 → 4) + ReLU

W^1 shape: torch.Size([4, 3]) (out_features × in_features)

b^1 shape: torch.Size([4])

$z^1 = x \cdot W^1 + b^1$:

Shape: torch.Size([2, 4])

Valores:

```

tensor([[ 1.5000,  3.1000,  5.2000,  1.3000],
        [ 3.3000,  7.6000, 12.4000,  3.1000]])

```

$a^1 = \text{ReLU}(z^1)$:

Shape: torch.Size([2, 4])

Valores:

```

tensor([[ 1.5000,  3.1000,  5.2000,  1.3000],
        [ 3.3000,  7.6000, 12.4000,  3.1000]])

```

CAPA 2: Lineal (4 → 2) - Capa de salida

$z^2 = a^1 \cdot W^2 + b^2$ (LOGITS):

Shape: torch.Size([2, 2])

Valores:

```

tensor([[2.3700, 2.9600],

```

```

[5.5200, 7.1000]])

Probabilidades (softmax):
  tensor([[0.3566, 0.6434],
         [0.1708, 0.8292]])
  Suma por fila: tensor([1., 1.]) (siempre suma 1)

Predicciones (clase con mayor probabilidad): tensor([1, 1])

```

1.7 6. Backpropagation

1.7.1 El problema de aprendizaje

Hemos visto cómo una red hace predicciones (forward pass). Pero, ¿cómo aprende la red? ¿Cómo ajustamos los millones de parámetros para que las predicciones mejoren?

La respuesta es **backpropagation** (propagación hacia atrás del error), combinada con **gradient descent** (descenso de gradiente).

1.7.2 Intuición

1. **Hacemos una predicción** (forward pass)
2. **Medimos el error** usando una función de pérdida
3. **Preguntamos:** ¿Cuánto contribuyó cada peso al error?
4. **Ajustamos los pesos** en la dirección que reduce el error

El paso 3 es el corazón de backpropagation: calcular la derivada parcial del error respecto a cada peso. Esto nos dice: - **Signo:** ¿Debemos aumentar o disminuir este peso? - **Magnitud:** ¿Cuánto afecta este peso al error?

1.7.3 La regla de la cadena

Backpropagation es simplemente una aplicación eficiente de la **regla de la cadena** del cálculo.

Si $L = f(g(h(x)))$, entonces:

$$\frac{dL}{dx} = \frac{dL}{df} \cdot \frac{df}{dg} \cdot \frac{dg}{dh} \cdot \frac{dh}{dx}$$

En una red neuronal, L es la pérdida final, y queremos calcular $\frac{\partial L}{\partial w}$ para cada peso w .

1.7.4 Proceso de backpropagation

El algoritmo procede en orden inverso al forward pass:

1. Calcular la pérdida $L = \mathcal{L}(\hat{y}, y)$
2. Calcular $\frac{\partial L}{\partial \mathbf{z}^{[L]}}$ (gradiente en la última capa)
3. Para cada capa $l = L, L-1, \dots, 1$:
 - Calcular $\frac{\partial L}{\partial \mathbf{W}^{[l]}}$ y $\frac{\partial L}{\partial \mathbf{b}^{[l]}}$
 - Propagar el gradiente hacia atrás: $\frac{\partial L}{\partial \mathbf{z}^{[l-1]}}$

```
[7]: # Demostración de autograd en PyTorch
import torch

print("=== AUTOGRAD: DIFERENCIACIÓN AUTOMÁTICA ===")
print()

# Crear tensores con requires_grad=True para rastrear operaciones
x = torch.tensor([2.0], requires_grad=True)
w = torch.tensor([3.0], requires_grad=True)
b = torch.tensor([1.0], requires_grad=True)

print("Variables:")
print(f"  x = {x.item()} (entrada)")
print(f"  w = {w.item()} (peso)")
print(f"  b = {b.item()} (bias)")
print()

# Forward pass:  $y = wx + b$ 
y = w * x + b
print(f"Forward:  $y = w \cdot x + b = \{w.item()\} \cdot \{x.item()\} + \{b.item()\} = \{y.item()\}$ ")

# Función de pérdida:  $L = y^2$  (ejemplo simple)
L = y ** 2
print(f"Loss:  $L = y^2 = \{y.item()\}^2 = \{L.item()\}$ ")
print()

# Backward pass: calcular todos los gradientes
L.backward()

print("=== GRADIENTES (calculados automáticamente) ===")
print()

# Verificación manual:
#  $L = y^2 = (wx + b)^2$ 
#  $L/y = 2y = 2(wx + b)$ 
#  $L/w = L/y \cdot y/w = 2y \cdot x$ 
#  $L/x = L/y \cdot y/x = 2y \cdot w$ 
#  $L/b = L/y \cdot y/b = 2y \cdot 1$ 

print("Derivada de L respecto a cada variable:")
print()
print(f"L/ w =  $2y \cdot x = 2 \cdot \{y.item()\} \cdot \{x.item()\} = \{2 * y.item() * x.$   

 $\hookrightarrow item()\}$ ")
print(f"PyTorch calcula:  $w.grad = \{w.grad.item()\}$  ")
print()
print(f"L/ x =  $2y \cdot w = 2 \cdot \{y.item()\} \cdot \{w.item()\} = \{2 * y.item() * w.$   

 $\hookrightarrow item()\}$ ")
```

```

print(f"          PyTorch calcula: x.grad = {x.grad.item()} ")
print()
print(f"L/ b = 2y · 1 = 2 · {y.item()} = {2 * y.item()}")
print(f"          PyTorch calcula: b.grad = {b.grad.item()} ")

```

=== AUTOGRAD: DIFERENCIACIÓN AUTOMÁTICA ===

Variables:

```

x = 2.0 (entrada)
w = 3.0 (peso)
b = 1.0 (bias)

```

Forward: $y = w \cdot x + b = 3.0 \cdot 2.0 + 1.0 = 7.0$

Loss: $L = y^2 = 7.0^2 = 49.0$

=== GRADIENTES (calculados automáticamente) ===

Derivada de L respecto a cada variable:

```

L/ w = 2y · x = 2 · 7.0 · 2.0 = 28.0
      PyTorch calcula: w.grad = 28.0

```

```

L/ x = 2y · w = 2 · 7.0 · 3.0 = 42.0
      PyTorch calcula: x.grad = 42.0

```

```

L/ b = 2y · 1 = 2 · 7.0 = 14.0
      PyTorch calcula: b.grad = 14.0

```

```

[8]: # Ejemplo con una red completa
import torch
import torch.nn as nn

print("=== BACKPROPAGATION EN UNA RED NEURONAL ===")
print()

# Red simple: 3 → 4 → 2
modelo = nn.Sequential(
    nn.Linear(3, 4),
    nn.ReLU(),
    nn.Linear(4, 2)
)

# Datos de ejemplo
x = torch.randn(1, 3) # 1 muestra, 3 features
y_true = torch.tensor([1]) # Clase verdadera

print(f"Entrada: {x}")

```

```

print(f"Clase real: {y_true.item()}")
print()

# Forward pass
y_pred = modelo(x)
print(f"Predicción (logits): {y_pred}")
print(f"Clase predicha: {y_pred.argmax().item()}")

# Calcular pérdida
loss_fn = nn.CrossEntropyLoss()
loss = loss_fn(y_pred, y_true)
print(f"\nPérdida: {loss.item():.4f}")

# Antes de backward, los gradientes son None
print("\nAntes de backward():")
for name, param in modelo.named_parameters():
    print(f" {name}.grad = {param.grad}")

# Backward pass: calcula gradientes
loss.backward()

# Después de backward, tenemos los gradientes
print("\nDespués de backward():")
for name, param in modelo.named_parameters():
    grad_norm = param.grad.norm().item()
    print(f" {name}: shape {list(param.grad.shape)}, norma del gradiente: ↵
↵{grad_norm:.4f}")

```

=== BACKPROPAGATION EN UNA RED NEURONAL ===

Entrada: tensor([[0.2615, -0.3309, 1.7900]])
Clase real: 1

Predicción (logits): tensor([[-0.2412, -0.0777]], grad_fn=<AddmmBackward0>)
Clase predicha: 1

Pérdida: 0.6147

Antes de backward():
 0.weight.grad = None
 0.bias.grad = None
 2.weight.grad = None
 2.bias.grad = None

Después de backward():
 0.weight: shape [4, 3], norma del gradiente: 0.0000
 0.bias: shape [4], norma del gradiente: 0.0000
 2.weight: shape [2, 4], norma del gradiente: 0.0000

2.bias: shape [2], norma del gradiente: 0.6494

1.7.5 Gradient Descent: Actualización de pesos

Una vez que tenemos los gradientes, actualizamos los pesos usando **gradient descent**:

$$\mathbf{w}_{\text{nuevo}} = \mathbf{w}_{\text{actual}} - \eta \cdot \frac{\partial L}{\partial \mathbf{w}}$$

Donde η (eta) es el **learning rate** (tasa de aprendizaje), un hiperparámetro crucial:

- **Muy pequeño**: El aprendizaje es lento; puede quedarse atascado en mínimos locales.
 - **Muy grande**: El aprendizaje es inestable; puede divergir (la pérdida aumenta).
 - **Valor típico**: Entre 0.0001 y 0.1, dependiendo del problema y optimizador.
-

1.8 7. Funciones de Pérdida

1.8.1 Definición y propósito

La **función de pérdida** (loss function) o **función de coste** (cost function) mide la discrepancia entre las predicciones del modelo y los valores reales. Es la métrica que el algoritmo de optimización intenta minimizar.

Una buena función de pérdida debe: 1. Ser **diferenciable** (para poder calcular gradientes) 2. Tener un **mínimo** cuando las predicciones son perfectas 3. **Penalizar** predicciones incorrectas de forma proporcional a su gravedad

1.8.2 Funciones de pérdida para REGRESIÓN

En problemas de regresión, predecimos un valor numérico continuo.

Mean Squared Error (MSE) - Error Cuadrático Medio

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **Penaliza más los errores grandes** (por el cuadrado)
- **Sensible a outliers**: Un solo valor atípico puede dominar la pérdida
- **Derivada simple**: $\frac{\partial \text{MSE}}{\partial \hat{y}_i} = \frac{2}{n} (\hat{y}_i - y_i)$

Mean Absolute Error (MAE) - Error Absoluto Medio

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- **Penalización lineal**: Todos los errores cuentan igual por unidad
- **Más robusto a outliers** que MSE
- **Derivada no suave** en cero (pero usable en la práctica)

```
[9]: import torch
import torch.nn as nn

print("=== FUNCIONES DE PÉRDIDA PARA REGRESIÓN ===")
print()

# Datos de ejemplo
y_real = torch.tensor([2.5, 0.0, 2.0, 8.0])
y_pred = torch.tensor([3.0, -0.5, 2.0, 7.5])

print(f"Valores reales:      {y_real.tolist()}")
print(f"Predicciones:        {y_pred.tolist()}")
print(f"Diferencias:         {(y_pred - y_real).tolist()}")
print()

# MSE
mse_loss = nn.MSELoss()
mse = mse_loss(y_pred, y_real)
print(f"MSE =  $\Sigma(y - \hat{y})^2 / n = {mse.item():.4f}")$ ")

# Cálculo manual
mse_manual = ((y_pred - y_real) ** 2).mean()
print(f"Verificación manual: {mse_manual.item():.4f}")
print()

# MAE
mae_loss = nn.L1Loss()
mae = mae_loss(y_pred, y_real)
print(f"MAE =  $\Sigma|y - \hat{y}| / n = {mae.item():.4f}")$ ")

# Cálculo manual
mae_manual = (y_pred - y_real).abs().mean()
print(f"Verificación manual: {mae_manual.item():.4f}")
```

=== FUNCIONES DE PÉRDIDA PARA REGRESIÓN ===

Valores reales: [2.5, 0.0, 2.0, 8.0]
 Predicciones: [3.0, -0.5, 2.0, 7.5]
 Diferencias: [0.5, -0.5, 0.0, -0.5]

MSE = $\Sigma(y - \hat{y})^2 / n = 0.1875$
 Verificación manual: 0.1875

MAE = $\Sigma|y - \hat{y}| / n = 0.3750$
 Verificación manual: 0.3750

1.8.3 Funciones de pérdida para CLASIFICACIÓN

Binary Cross-Entropy (BCE) - Clasificación Binaria Para problemas con **2** clases (0 o 1):

$$\text{BCE} = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Donde $\hat{y}_i$ es la probabilidad predicha de la clase 1.

Interpretación: - Si $y_i = 1$: El término relevante es $-\log(\hat{y}_i)$. Cuanto más cerca de 1 esté $\hat{y}_i$, menor la pérdida. - Si $y_i = 0$: El término relevante es $-\log(1 - \hat{y}_i)$. Cuanto más cerca de 0 esté $\hat{y}_i$, menor la pérdida.

Cross-Entropy - Clasificación Multiclase Para problemas con **C** clases:

$$\text{CE} = -\frac{1}{n} \sum_{i=1}^n \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c})$$

Donde: - $y_{i,c}$ es 1 si la muestra i pertenece a la clase c , 0 en caso contrario (one-hot encoding) - $\hat{y}_{i,c}$ es la probabilidad predicha de que la muestra i pertenezca a la clase c

En PyTorch, `nn.CrossEntropyLoss` combina **softmax** + **negative log-likelihood**, por lo que esperamos **logits** (valores sin normalizar) como entrada, no probabilidades.

```
[10]: print("=== FUNCIONES DE PÉRDIDA PARA CLASIFICACIÓN ===")
print()

# ===== CLASIFICACIÓN BINARIA =====
print("--- Clasificación Binaria (2 clases) ---")
print()

y_real_bin = torch.tensor([1.0, 0.0, 1.0, 1.0]) # Clases reales
# Logits: valores antes de aplicar sigmoid
logits_bin = torch.tensor([2.0, -1.0, 0.5, 3.0])

# BCEWithLogitsLoss aplica sigmoid internamente (más estable numéricamente)
bce_loss = nn.BCEWithLogitsLoss()
bce = bce_loss(logits_bin, y_real_bin)

print(f"Clases reales:      {y_real_bin.tolist()}")
print(f"Logits:              {logits_bin.tolist()}")
print(f"Probabilidades ( ):    {torch.sigmoid(logits_bin).tolist()}")
print(f"BCE Loss:              {bce.item():.4f}")
print()

# ===== CLASIFICACIÓN MULTICLASE =====
print("--- Clasificación Multiclase (3 clases) ---")
print()
```

```

# Clases reales (índices, no one-hot)
y_real_multi = torch.tensor([0, 2, 1]) # 3 muestras, clases 0, 2, 1

# Logits: (batch_size, num_classes)
logits_multi = torch.tensor([
    [2.0, 0.5, 0.3], # Predice clase 0
    [0.1, 0.2, 2.5], # Predice clase 2
    [0.5, 1.8, 0.3]  # Predice clase 1
])

ce_loss = nn.CrossEntropyLoss()
ce = ce_loss(logits_multi, y_real_multi)

print(f"Clases reales:    {y_real_multi.tolist()}")
print(f"Logits:\n{logits_multi}")
print(f"\nProbabilidades (softmax):")
probs = torch.softmax(logits_multi, dim=1)
print(f"{probs}")
print(f"\nPredicciones:    {logits_multi.argmax(dim=1).tolist()}")
print(f"CrossEntropy Loss: {ce.item():.4f}")

```

=== FUNCIONES DE PÉRDIDA PARA CLASIFICACIÓN ===

--- Clasificación Binaria (2 clases) ---

```

Clases reales:      [1.0, 0.0, 1.0, 1.0]
Logits:              [2.0, -1.0, 0.5, 3.0]
Probabilidades ():   [0.8807970285415649, 0.2689414322376251,
0.622459352016449, 0.9525741338729858]
BCE Loss:            0.2407

```

--- Clasificación Multiclase (3 clases) ---

```

Clases reales:      [0, 2, 1]
Logits:
tensor([[2.0000, 0.5000, 0.3000],
        [0.1000, 0.2000, 2.5000],
        [0.5000, 1.8000, 0.3000]])

```

```

Probabilidades (softmax):
tensor([[0.7113, 0.1587, 0.1299],
        [0.0762, 0.0842, 0.8396],
        [0.1822, 0.6686, 0.1492]])

```

```

Predicciones:      [0, 2, 1]
CrossEntropy Loss: 0.3060

```

1.8.4 Resumen: Selección de función de pérdida

Tipo de problema	Función de pérdida	Clase PyTorch	Activación salida
Regresión	MSE	<code>nn.MSELoss()</code>	Ninguna (lineal)
Regresión (robusta)	MAE	<code>nn.L1Loss()</code>	Ninguna (lineal)
Clasificación binaria	BCE	<code>nn.BCEWithLogitsLoss()</code>	Ninguna (logits)
Clasificación multiclase	Cross-Entropy	<code>nn.CrossEntropyLoss()</code>	Ninguna (logits)

Nota importante: En PyTorch, las funciones `BCEWithLogitsLoss` y `CrossEntropyLoss` esperan **logits** (valores sin normalizar), no probabilidades. Aplican internamente sigmoid/softmax por estabilidad numérica.

1.9 Resumen del Notebook

En este notebook hemos cubierto los fundamentos teóricos del Deep Learning:

Concepto	Descripción clave
Deep Learning	Redes neuronales con múltiples capas que aprenden representaciones jerárquicas
Perceptrón	Unidad básica: $y = f(\mathbf{w}^T \mathbf{x} + b)$
Funciones de activación	Introducen no-linealidad; ReLU es la más usada en capas ocultas
MLP	Red de múltiples capas fully connected
Forward pass	Flujo de datos entrada $\rightarrow$ predicción
Backpropagation	Cálculo eficiente de gradientes usando la regla de la cadena
Gradient descent	Actualización de pesos: $w \leftarrow w - \eta \nabla L$
Funciones de pérdida	MSE para regresión, Cross-Entropy para clasificación

1.9.1 Siguiente paso

En el siguiente notebook ([DL_02_pytorch_basico.ipynb](#)), aprenderemos a implementar estos conceptos usando PyTorch, la biblioteca de Deep Learning más popular.